PYTHON PROGRAMMING – ASSIGNMENT 1

Design, Implement and Validate a Payment Calculator for a Village Milk Co-operative

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences, Chennai

Student Name: Muthupandi C

Register Number: 192565037

Course Code: CSA0805

Submission Date: [22.10.2026]

Contents

- Task 1 – Understand the Problem and Identify the Data
- Task 1 – Alternative Design and Complexity
- Task 2 – Values and Types
- Task 2 – Alternative Money Representations
- Task 3 – Variables and Constants
- Task 4 – Correct Expressions
- Task 5 – Read Input and Convert Types
- Task 6 – Organise Logic into Functions
- Task 7 – Build a Module
- Task 8 – Algorithmic Problem Solving
- Task 9 – Implement and Test the Calculator
- Task 10 – Compare Expectations with Execution
- Task 11 – Recommend and Defend the Final Design
- Task 12 – Re-evaluate and Adapt the Design
- Submission Checklist

Task 1 – Understand the Problem and Identify the Data

Inputs: farmer ID (text, because it may contain letters or leading zeroes), litres (fractional numeric quantity), fat percentage and SNF percentage (fractional readings), shift (text such as M/E), and date/ten-day period (needed to group payments). Outputs: calculated rate per litre, amount for each delivery, and the sum for each farmer across the ten-day period.

Named constants: BASE_RATE = Rs 30.00, REFERENCE_FAT = 3.5%, REFERENCE_SNF = 8.5%, FAT_STEP_VALUE = Rs 0.40 per 0.1% fat step, SNF_STEP_VALUE = Rs 0.30 per 0.1% SNF step, STEP_SIZE = 0.1, MIN_FAT = 2.0%, MAX_FAT = 7.0%. These are policy values and should be changed in one place only.

Types: farmer ID and shift are strings; litres, fat and SNF are fractional values; payment values should use Decimal or integer paise to avoid binary floating-point errors; entry count can be an integer; validation outcomes are Boolean.

Rules: quantity must be positive; fat must be within 2.0–7.0%; fat and SNF must be numeric; shift must be M or E; invalid values must not be paid; every payment must be rounded to the nearest paisa using a stated rule. SNF range is not specified in the brief, so the implementation assumes 0–20% provisionally and marks this for confirmation. The rate formula uses nearest whole 0.1 percentage-point steps.

Assumptions: readings are percentage values (e.g. 4.2 means 4.2%); each delivery is paid separately to two decimal places; ten-day total is the sum of rounded delivery payments; one farmer may have multiple deliveries and shifts.

Task 1 – Alternative Design and Complexity

Separate variables such as litres1, fat1, snf1 through litres300, fat300, snf300 create repetitive code and make loops, validation, and scaling difficult. A list of records such as (farmer_id, litres, fat, snf, shift) allows one loop to process n entries. Both designs need O(n) data space if every entry is retained and O(n) processing time, but the record-list design is maintainable and supports growth to 6,000 or 150,000 entries. If only a running total is needed, the running-total approach uses O(1) additional space.

Task 2 – Values and Types

Interpreter examples:

```
>>> type(10)                    # int
<class 'int'>
>>> type(10.0)                   # float
<class 'float'>
>>> type("10")                   # str
<class 'str'>
>>> type(True)                   # bool
<class 'bool'>
>>> type([10, 8.5])               # list
<class 'list'>
>>> 7 / 2
3.5
>>> 7 // 2
```

```
3
>>> 7 % 2
1
>>> -7 // 2
-4
>>> 0.1 + 0.2 == 0.3
False
```

`/` returns ordinary division; `//` floors toward negative infinity, so -7 // 2 is -4; `%` returns the remainder. Binary floats cannot represent many decimal fractions exactly, which explains why 0.1 + 0.2 does not equal the stored representation of 0.3. Money should use integer paise or Decimal constructed from strings. `round()` can be appropriate only with a clearly chosen rounding policy; it does not make every intermediate float exact.

Completed TODO:

```
def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total
```

`Decimal("33.40")` preserves the decimal text exactly. `Decimal(float("0.1"))` instead imports the float's binary approximation, so its result contains extra digits. For n values, summing is O(n) time; the running total uses O(1) additional space. A Decimal object uses more memory than a float; `sys.getsizeof()` can be used on the actual Python installation to measure it.

Task 2 – Alternative Money Representations

Integer paise: convert each amount to an integer number of paise and sum integers. This is exact when all amounts are already expressed in whole paise; it is generally efficient and has O(n) time and O(1) accumulator space. Decimal: sum decimal strings and round only at the defined payment boundary; this is exact for decimal inputs and transparent for financial rules. Float plus round after every addition is not a robust solution because binary representation error remains and repeated rounding can alter totals. Measure memory using `sys.getsizeof(0.1)` and `sys.getsizeof(Decimal('0.1'))` on the target interpreter; exact sizes depend on Python build/version.

Task 3 – Variables and Constants

Python identifiers should be descriptive, begin with a letter or underscore, and contain letters, digits and underscores. Constants are conventionally written in UPPER_CASE; Python does not enforce immutable module constants, so the convention is followed by programmers. Example: BASE_RATE, FAT_STEP_VALUE, SNF_STEP_VALUE, REFERENCE_FAT, REFERENCE_SNF and STEP_SIZE. Python is dynamically typed: `value = 10` binds an integer, then `value = 'ten'` rebinds the name to a string. Multiple assignment supports `a, b = b, a` to swap values without a temporary variable.

If magic numbers are repeated in k expressions, a rate change may require k edits and can be applied inconsistently. A named constant requires one edit. A dictionary such as RATE_CARD can group related values and make seasonal selection convenient. Runtime and memory differences are negligible for this small rate card; maintainability and correctness are the main reasons to use named constants.

Task 4 – Correct Expressions

Fat steps = round((fat - REFERENCE_FAT) / STEP_SIZE)
SNF steps = round((snf - REFERENCE_SNF) / STEP_SIZE)
Rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
Amount = litres * rate
The implementation uses Decimal and ROUND_HALF_UP for explicit financial rounding.

Unexpected examples:
1. `(4.2 - 3.5) / 0.1` can display `7.000000000000001` because 4.2, 3.5 and 0.1 are not all exactly represented as binary floats.
2. `0.1 + 0.2 == 0.3` is False because the stored binary approximations differ.
3. `-7 // 2` is -4, while `int(-7 / 2)` is -3: floor and truncation differ for negative values.
4. `round(2.5)` is 2 in Python because the built-in uses ties-to-even for exact halfway cases.

For fat=4.2, the ideal step count is +7; for fat=3.1 it is -4. `int()` truncates toward zero and `math.floor()` rounds down, so small representation errors or non-integer step values can change results. Each operation is O(1) time and O(1) space. Comparisons can be chained (`2.0 <= fat <= 7.0`), and Boolean `and` combines validation conditions.

Task 5 – Read Input and Convert Types

`input()` always returns text. Use `int()` for whole-number identifiers/counts when suitable and `float()` or, preferably for financial calculations, `Decimal()` for fractional values. `float('eight')` and `float('8,5')` raise ValueError under the expected decimal-point format. `float(' 8.5 ')` accepts surrounding whitespace; calling `.strip()` first makes the intent clear. A try/except ValueError loop provides direct and reliable validation. String checks alone are easy to get wrong because signs and decimal points require careful rules. Output can be aligned using f-strings, e.g. `f'{amount:8.2f}'`. Parsing and validating a value is O(1) relative to the number of entries; processing n entries is O(n).

Task 6 – Organise Logic into Functions

The solution uses small functions: `rate_per_litre(fat, snf)` returns a rate; `payout(litres, fat, snf)` returns one delivery amount; `ten_day_total(payments)` returns a total; and `validate_entry(...)` checks input rules. Returning a value makes the result reusable and testable, while printing only displays it. Function parameters are local names. Module constants can be read inside functions, but modifying global policy values from a function should be avoided. Docstrings describe purpose, inputs, and returned values.

A single long script may initially contain fewer function-call operations, but repeated logic becomes difficult to test and maintain. Lambda expressions are useful for short anonymous expressions, not for multi-step validation and financial rules. Function calls add a small constant overhead; overall processing remains O(n) for n entries, so the difference is usually minor compared with clarity and reuse.

Task 7 – Build a Module

`milkrate.py` stores the rate card and calculation functions. `import milkrate` uses qualified names such as `milkrate.payout`; `from milkrate import payout` imports one name directly; `import milkrate as mr` uses an alias. `if __name__ == '__main__':` runs the demonstration only when the file is executed directly. When imported, `__name__` is the module name (`milkrate`), so the demonstration is skipped. Without the guard, the

demonstration would run on import and create unwanted output or side effects. Python searches the current/script directory and entries in `sys.path`; standard modules include `decimal` and `math`.

If code were copied into every script, each copy would need updating after a rate revision. A shared module allows one maintained implementation. Importing has a one-time startup cost; Python caches loaded modules in `sys.modules`, so repeated imports generally reuse the module object. For n payments, calculation remains O(n).

Task 8 – Algorithmic Problem Solving

Pseudocode:

```
START
SET total = 0.00
FOR each delivery entry
   READ farmer_id, litres, fat, snf, shift
   IF any required value is invalid THEN
      DISPLAY validation error
      CONTINUE to next entry
   END IF
   CALCULATE fat_steps and snf_steps
   CALCULATE rate = base + fat_steps × fat_step_value + snf_steps × snf_step_value
   ROUND rate to 2 decimal places
   CALCULATE amount = litres × rate
   ROUND amount to 2 decimal places
   ADD amount to total
   DISPLAY rate and amount
END FOR
DISPLAY total
END
```

Validation flow:
Start → Read entry → Numeric conversion successful?
No → Show error → Read again
Yes → Quantity > 0 and fat in 2.0–7.0 and shift valid?
No → Show error → Read again
Yes → Calculate rate and amount → Store/accumulate result → Next entry/end.

Hand trace:
Entry 1: 10 L, fat 3.5, SNF 8.5 → rate Rs 30.00 → Rs 300.00.
Entry 2: 8.5 L, fat 4.2, SNF 8.7 → 7 fat steps and 2 SNF steps → rate Rs 33.40 → Rs 283.90.
Entry 3: 12 L, fat 3.1, SNF 8.3 → -4 fat steps and -2 SNF steps → rate Rs 27.80 → Rs 333.60.
Total = Rs 917.50.

Storing records and adding at the end takes O(n) time and O(n) space, and is useful for audits, corrections, and ten-day grouping. A running total also takes O(n) time but O(1) extra space; use it when past records need not be revisited. A production system normally stores auditable entries even if it also maintains a running total.

Task 9 – Implement and Test the Calculator

The program validates entries before calculation and has automated assertions. Sample test cases:
1. 10 L, fat 3.5, SNF 8.5 → expected Rs 300.00; actual Rs 300.00.
2. 8.5 L, fat 4.2, SNF 8.7 → expected Rs 283.90; actual Rs 283.90.
3. 12 L, fat 3.1, SNF 8.3 → expected Rs 333.60; actual Rs 333.60.
4. Fat 2.0, SNF 8.5 → rate Rs 24.00 (lower fat boundary).
5. Fat 7.0, SNF 8.5 → rate Rs 44.00 (upper fat boundary).
6. Negative quantity → rejected.
7. Fat 7.1 → rejected.
8. Text quantity `eight` → rejected during numeric conversion.
9. Empty farmer ID or invalid shift → rejected.

Five-entry hand total should be computed by adding each rounded payment. Automated `assert` tests can be rerun with one command after every change and are repeatable; manual testing requires repeated typing and is slower and more error-prone. The included test file runs core assertions. Add screenshots from your own execution to provide required evidence.

Task 10 – Compare Expectations with Execution

`Task10/task10_benchmark.py` measures float and Decimal summation for 300, 6,000 and 150,000 payments with `time.perf_counter()`. It prints the actual totals, difference and elapsed times on the machine where it is run. Do not treat timing numbers from another computer as universal: processor, Python version, and system load affect them. Float may accumulate a tiny representation error; Decimal is slower but retains decimal arithmetic when constructed from strings.

The correct accounting rule is to round each payable delivery to the nearest paisa, then sum those rounded payments for the ten-day bank total. Rounding only the final aggregate can disagree where individual delivery amounts have fractions of a paisa. A dictionary rate cache can provide expected O(1) lookup for a previously seen (fat, SNF) pair, but it adds O(k) memory for k unique pairs and requires a stable key/rounding policy. Recalculation is already O(1) per rate, so caching should be adopted only if measurements show a meaningful benefit.

Task 11 – Recommend and Defend the Final Design

Use Decimal constructed from strings for analyser readings and monetary values; specify ROUND_HALF_UP to the nearest paisa at the payment boundary. Keep rates in one rate-card module or a JSON file. A rate revision should be reviewed, versioned, tested, and deployed as one change rather than edited in multiple expressions. The JSON approach permits a non-programmer to update policy values but requires validation of the file, error handling, and a record of which rate-card version applied to each payment.

Limitations: the supplied exercise does not define a valid SNF range, tie-breaking policy for analyser steps, handling of corrections, database persistence, concurrent access, or official seasonal calendar rules. The implementation states assumptions rather than presenting them as official policy. At district-union scale (150,000 farmers), a single pass remains O(n), but a production system should use database transactions, audit logs, access controls, backups, and batch processing. Run Task 10 on the target computer and include the actual measured values in the final submission.

Task 12 – Re-evaluate and Adapt the Design

Decision: MODIFY the original design. The original constants and functions remain useful, but a season must now select a rate card and a separate bonus rule must be applied. The analyser can provide two decimal places, so step counting should explicitly divide the reading difference by 0.1 and round to a whole step under a documented rule. Decimal arithmetic avoids binary-float step surprises.

Changed requirements: April–June base rate = Rs 32.00; fat step = Rs 0.45; SNF step remains Rs 0.30 by assumption because the brief changes only base and fat rates; bonus = Rs 0.50 per litre if ten-day quantity is strictly greater than 500 litres. The rate card is passed through a `season` parameter. The bonus is based on the farmer's ten-day total, not the current delivery alone.

Worked tests:
- Regular season, 10 L at reference values, no bonus → rate Rs 30.00, payout Rs 300.00.
- April–June, 10 L at reference values, 300 L ten-day total → rate Rs 32.00, payout Rs 320.00.
- April–June, fat 4.25 and SNF 8.55: half-step readings require a declared tie rule. With ROUND_HALF_UP, fat difference 0.75 / 0.1 = 7.5 steps rounds to 8; SNF difference 0.05 / 0.1 = 0.5 rounds to 1. Rate = 32.00 + (8 × 0.45) + (1 × 0.30) = Rs 35.90. For 20 L, base payout = Rs 718.00; if ten-day total is 520 L, bonus = Rs 10.00; total = Rs 728.00.
- Same 20 L delivery with ten-day total exactly 500 L → no bonus; payout Rs 718.00.

Evidence-based defence: the revised program keeps reusable calculation functions, introduces season selection and bonus logic, and retains validation. The trade-off is more configuration and the need to confirm the tie rule and unchanged SNF step value with the co-operative. Run Task 12 and include its console output screenshot.